

Earth Embeddings as Products

Taxonomy, Ecosystem, and Standardized Access

Heng Fang* — KTH Royal Institute of Technology

Adam J. Stewart* — Technical University of Munich

Isaac Corley — Taylor Geospatial

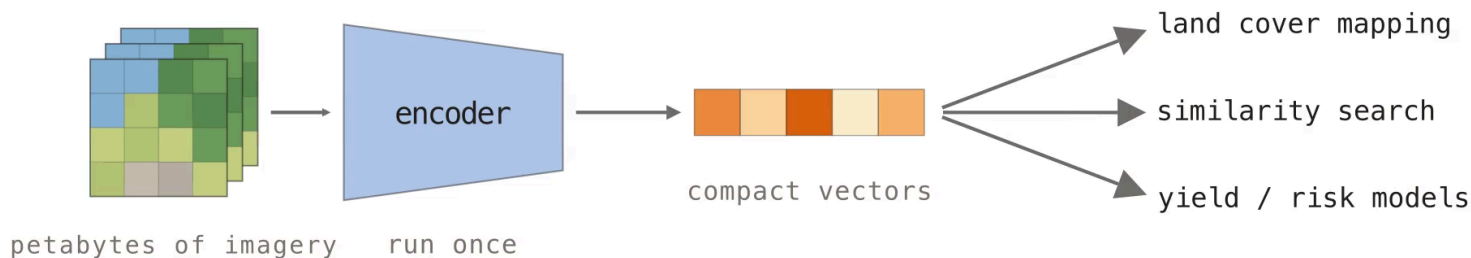
Xiao Xiang Zhu — Technical University of Munich

Hossein Azizpour — KTH Royal Institute of Technology

*equal contribution · since expanded into the book chapter *Earth Embeddings* (arXiv, August 2026)

Embedding products move model inference out of the user's workflow

NASA's EOSDIS alone holds **178.7 PB** of imagery and grows by 160 TB per day. Foundation models can summarize this archive, but every team re-downloads the same pixels and re-pays the same preprocessing and GPU inference. Embedding products run the model once and ship the vectors as reusable data.



The model is not the map

The Model is Not the Map

Please Look at the Data



CHRISTOPHER REN
JUN 21, 2025



5



2



1

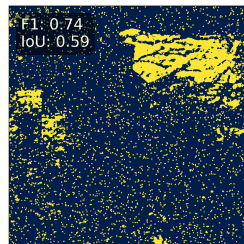
Share

TLDR;

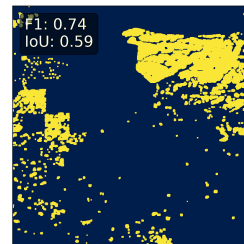
- Make maps, not ~~war~~ (just) benchmarks
- I made the Santa Fe Quartet as a geospatial equivalent to Anscombe's quartet
- If you are interested in building this kind of map-benchmark let's chat!

Four planted failure modes, **near-identical scores**, four visibly different maps. Ship the predictions and embeddings, not just the checkpoint — nobody knows a model better than the team that trained it.

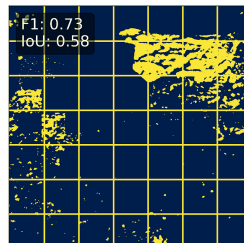
a) Random Salt



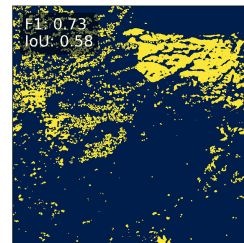
b) Neighbor Dilation



c) Tiling Issue



d) Class Confusion



A foundation model is not an embedding product

Foundation models (dynamic inference)

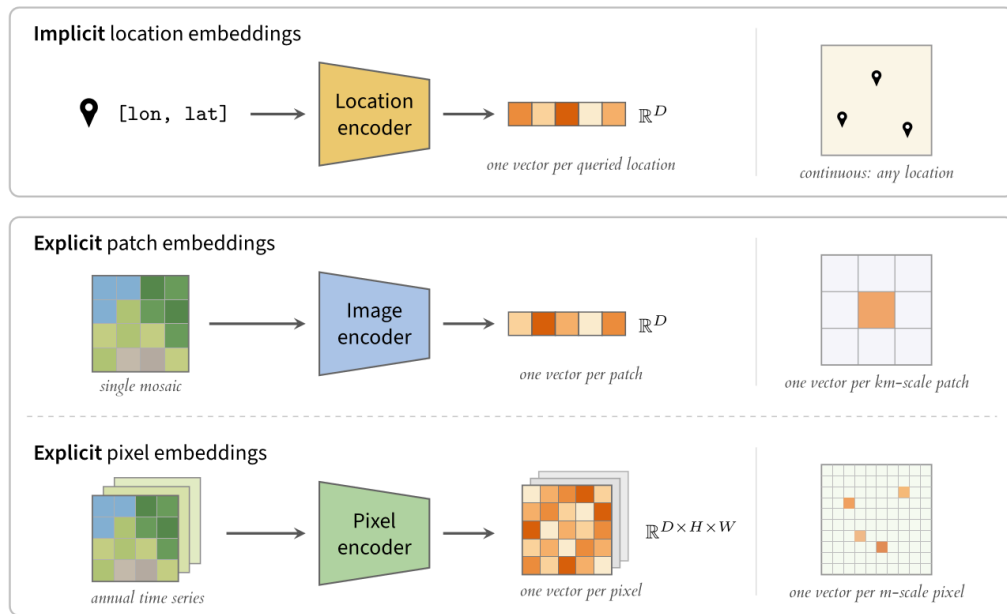
These are distributed as public weights (DOFA, OlmoEarth). Users manage preprocessing and GPU inference themselves. This is flexible but raises a hardware and engineering barrier.

Embedding products (static data)

These are distributed as pre-computed vector archives (Google Satellite Embedding). The assets are frozen and versioned, so analysis proceeds without the model or its compute.

A product is pinned to the data snapshot it was computed on, so **treating a product as a proxy for its model invites invalid generalization claims.**

Three families: location, patch, and pixel embeddings



Patch products summarize km-scale tiles for retrieval

All known patch embedding products as of July 2026. *sparse spatial or temporal coverage.

Product	Extent	Resolution	Years	Dim.	Dtype
MOSAIKS (2021–25)	USA / Global	1 km / 0.01°	2018 / 2019	8192 / 4000	float32
Clay v0 / v1.5 (2024–26)	USA / Global	154 m – 5.12 km	2010–2025*	768–1024	float32
Major TOM (2024–26)	Global*	120 m – 3.84 km	2016–2024*	256–2048	float32
Earth Index (2025)	Global	320 m	2024	384	float32
Copernicus-Embed (2025)	Global	0.25°	2021	768	float32

Major TOM is the largest family, with more than a dozen model variants on one shared grid. Every product stores float32. One vector summarizes an entire mosaic tile, which suits retrieval more than dense mapping.

Pixel products store one vector per pixel for each year

All known pixel embedding products as of July 2026. Every product is built from annual time series. *sparse coverage.

Product	Extent	Resolution	Years	Dim.	Dtype
Presto Embeddings (2025)	Togo	10 m	2019–2020	128	uint16
Tessera Embeddings (2025)	Global*	10 m	2017–2025*	128	int8 → float32
Google Satellite Embedding (2025)	Global	10 m	2017–2025	64	int8 → float64
Embedded Seamless Data (2026)	Global	30 m	2000–2024	12	uint16 → float32

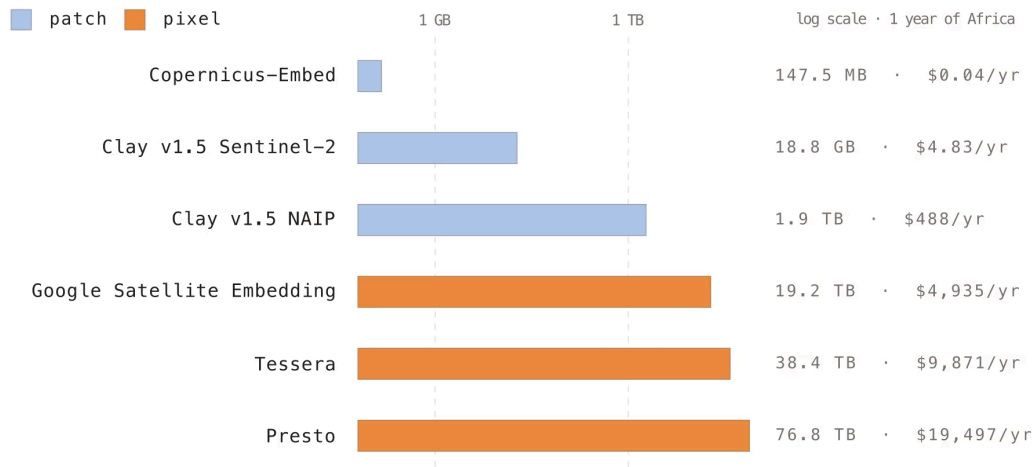
Google Satellite Embedding covers the full Sentinel era at 64 dimensions. **Embedded Seamless Data** trades resolution for 25 years of Landsat history. Storage pressure drives the int8 and uint16 dtypes.

Each product ships with its own formats, grids, and hosting

- Products are scattered across **Source Cooperative** (Clay, Earth Index), Hugging Face (Major TOM, Copernicus-Embed), Google Earth Engine (Google, Presto), and private university servers (Tessera).
- Formats range from **GeoParquet** to GeoTIFF with implicit CRS assumptions to raw NumPy arrays with no CRS or bounds ("numbers and a prayer").
- Each product defines its own **tiling grid** (Major TOM grid, MGRS, custom), so cross-product comparison starts with reprojection.
- **One flipped coordinate** in the Google Satellite Embedding rasters required patches to **GDAL, rasterio, and TorchGeo** before standard tools could read them.

Every team solves distribution independently. The integration tax is paid once per product, per user.

Storage for one continent-year spans five orders of magnitude



Patch products stay in the MB–GB range, while dense 10 m pixel products reach tens of TB. A single full download of Presto’s Africa coverage adds **~\$6.9k in egress** on top of storage.

Hosting and format choices decide whether a product is usable

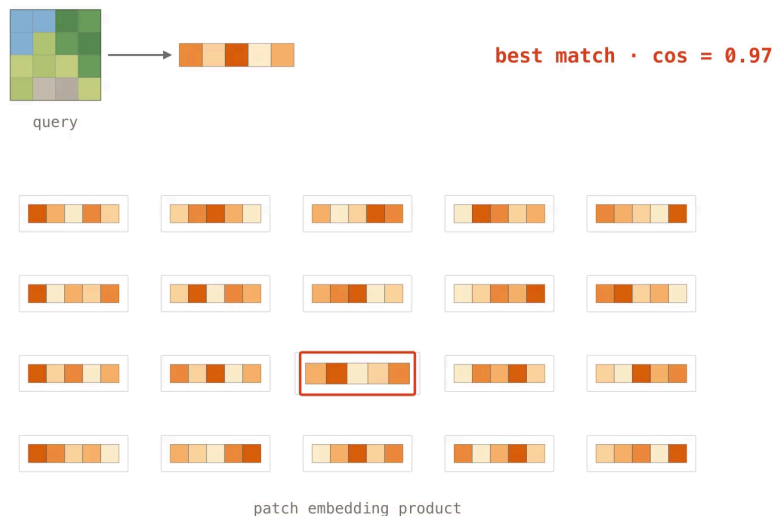
- **Hugging Face** enforces repo storage caps and API rate limits, so bulk pulls of TB-scale products throttle or fail.
- **Tessera** ships raw `.npy` tiles with coordinates and metadata in a sidecar file. NumPy supports no HTTP range requests, so reads pull whole tiles that a **COG or Zarr would stream** — GeoTIFF re-implemented without its streaming or metadata.
- **Google Satellite Embedding** was reachable only inside Earth Engine until **Taylor Geospatial** rehosted it on **Source Cooperative**, which adds free egress, an HTTP cross-region proxy, and CORS so browsers can stream it.
- Several products are so **sparse in space and time** that no common footprint exists for comparing them.

Patch products are reproducible, but pixel models are often proprietary

- **Clay, Earth Index, and Copernicus-Embed** release the full pipeline (code, weights, data, embeddings) under permissive licenses.
- **Major TOM's CC-BY-SA** pretraining data makes every derived embedding copyleft, which deters commercial users.
- **AlphaEarth Foundations and ESDNet** keep code and weights proprietary. The embeddings are CC-BY, but no one outside can regenerate or audit them.
- **No product ships checksummed data.** Upstream archives keep reprocessing imagery, so the exact training and inference inputs are already unrecoverable.

TorchGeo treats embeddings as first-class geospatial datasets

- TorchGeo has loaders for **every known embedding product**, plus the generating models and weights. **Presto and Tessera** were added upstream for this work.
- Reprojection, rasterization, spatiotemporal intersection, and sampling are built in.
- These workflows used to require **four or more repositories** and custom loaders. With TorchGeo each is about 20 lines of code.



Search and land cover mapping in TorchGeo

Search and retrieval: query by example

```
# torchgeo.datasets / torchgeo.models
earthindex = EarthIndexEmbeddings('data/ei')
s2 = Sentinel2('data/s2')
image = s2[xmin:xmax, ymin:ymax]['image'] / 10_000

model = vit_small_patch14_dinov2(
    ViTSmall14_DINOv2_Weights.SENTINEL2_ALL_SOFTCON)
embed = model(image)

cos = CosineSimilarity(dim=0)
for sample in iter(earthindex):
    sim = cos(embed, sample['embedding'])
    ... # keep the argmax, plot the match
```

Land cover mapping: crop types across Europe

```
# torchgeo.datasets / torchgeo.samplers
tessera = TesseraEmbeddings('data/tessera')
eurocrops = EuroCrops('data/ec', download=True)
dataset = tessera & eurocrops # intersection

train_roi = box(-10, 35, 10, 60) # West EU
test_roi = box(10, 35, 30, 60) # East EU
train_ds, test_ds = roi_split(
    dataset, [train_roi, test_roi])

# random patches → fit a k-NN / linear probe
# gridded patches → stitch a map of Europe
```

Embeddings improve mapping tasks but degrade under spatial transfer

Task	Embeddings	Finding
Cropland mapping, Togo	Presto, Google	Presto + Random Forest gives the best F1
Tree species, Dutch forest inventory	Presto, Google, Tessera	+2–9 points over hand-designed time-series features
Landslide susceptibility, TW·HK·IT	Google	64-d embeddings beat conventional conditioning factors
Poverty mapping, Sub-Saharan Africa	Google + GNN	large storage/preprocessing savings vs. raw Sentinel-2
Scene classification, EuroSAT-Embed	Google, Tessera, OlmoEarth	pooling choice cuts the geographic gap by >50%
Fusion across six tasks	Google, Tessera, GeoCLIP, SatCLIP	fused embeddings beat the best single model in 4 of 6

Performance drops under **spatial transfer**, and annual composites wash out sub-annual dynamics. Validate with geographic splits and simple baselines before trusting any product.

Ship cloud-native formats, in-file metadata, and int8 vectors

Embedding type	Format
Location / implicit	ONNX or PyTorch
Patch / region	GeoParquet
Pixel, snapshot	GeoTIFF or GeoZarr
Pixel, time series	GeoZarr

Ship a model card with sensors, temporal window, CRS and grid, dtype, quantization transform, and license. Store the metadata in the files, not the docs.

- **int8 quantization** has negligible accuracy cost. Google and Tessera already ship int8.
- **PCA to 64 dimensions with int8** gives 64× compression with <2% accuracy loss.
- **Binary quantization** adds another 32× and still recovers ~65% of float32 nearest neighbors, which is enough for candidate retrieval.
- Ship a **runnable benchmark**, not a private leaderboard.

Embeddings are usable today; standards are the remaining work

Pick the family by task: implicit for location context, patch for retrieval, pixel for dense mapping.

Validate under geographic splits against simple baselines. Open problems are ocean and atmosphere coverage, uncertainty layers, and shared benchmarks (identical models differ by more than ten points across papers).

Chapter arxiv.org/abs/2608.03410

Survey github.com/hfangcat/Awesome-Geospatial-Embeddings

Code github.com/torchgeo/torchgeo

Contact isaac.corley@taylorgeospatial.org